

The Mathematics and Algorithms of Texas Hold'em Poker

From Card Probabilities to Superhuman Play

Cathy Li

September 2026

Abstract

Texas Hold'em is the most strategically rich form of poker played at scale and, for the same reason, has served for three decades as the canonical testbed for decision making under uncertainty. This paper presents a self-contained treatment of the mathematical and algorithmic principles that underpin strong play. We begin with the combinatorial and probabilistic foundations of the game: hand frequencies, equity, pot odds, expected value, and the variance structure that dominates short-term results. We then develop the algorithmic toolkit in increasing order of sophistication: rule-based expert systems driven by hand strength, position, and pot odds; Monte Carlo equity estimators whose accuracy decays as $1/\sqrt{n}$; game-theoretic formulations of imperfect-information games in extensive form; counterfactual regret minimization (CFR) and its sampling and function-approximation variants, including CFR+ and Deep CFR; and reinforcement-learning approaches culminating in the superhuman systems DeepStack, Libratus, and Pluribus. We accompany the theory with a concrete implementation—a No-Limit Hold'em engine with five AI paradigms sharing one table—and report a 100,000-hand simulation study quantifying how the paradigms' win rates, aggression, and showdown statistics differ in practice. Throughout, emphasis is placed on rigorous statements, convergence guarantees, and the engineering compromises—state abstraction, action abstraction, and equilibrium approximation—that make theory tractable in a game whose full tree contains more game states than there are atoms in the observable universe.

Keywords: Texas Hold'em; imperfect-information games; Nash equilibrium; counterfactual regret minimization; Monte Carlo methods; reinforcement learning; abstraction.

Contents

1	Introduction	3
2	The Game of Texas Hold'em	3
2.1	Structure and terminology	3
2.2	Hand rankings	4
2.3	From cards to decisions	4
3	Probability Foundations	5
3.1	Combinatorics of the deck	5
3.2	Hand-category frequencies	5
3.3	Preflop equity	5
3.4	Conditional probability and reads	6
3.5	Pot odds, equity, and expected value	7
3.6	Variance, swings, and the Kelly criterion	9
4	Hand Evaluation, Monte Carlo, and Expert Systems	9
4.1	Exact hand evaluation	9
4.2	Monte Carlo equity estimation	10
4.3	Rule-based expert systems	10

5	Game-Theoretic Foundations	12
5.1	Extensive-form games with imperfect information	12
5.2	Nash equilibrium and its guarantees	13
5.3	Why perfect-information methods fail	13
5.4	A complete worked example: Kuhn poker	14
6	Counterfactual Regret Minimization	15
6.1	Regret and regret matching	15
6.2	The CFR decomposition	16
6.3	Abstraction: making the game fit	17
6.4	Deep CFR: regrets as a learned function	18
7	Reinforcement Learning in Imperfect-Information Games	18
7.1	Markov decision processes and Q-learning	18
7.2	Where the substrate breaks	19
7.3	The superhuman architectures	20
7.4	Our RL agent: tabular Q-learning with an opponent pool	20
8	Experimental Study: Five Paradigms at One Table	21
8.1	Setup and methodology	21
8.2	Results	22
8.3	Style analysis	22
8.4	Interpretation: why did the equilibrium agent lose?	24
8.5	Threats to validity	25
9	Conclusion	25

1 Introduction

Poker occupies a unique position at the intersection of probability theory, game theory, and artificial intelligence. Unlike chess or Go, it is a game of *imperfect information*: each player sees only a fragment of the true state, and the deliberate concealment of cards creates strategic phenomena—bluffing, trapping, and the delicate balance between exploiting weak opponents and remaining unexploitable oneself—that have no analogue in perfect-information games. It is also a game of chance at the level of a single hand yet a game of skill in the long run, a duality that makes it an ideal laboratory for studying decision making under uncertainty. These properties led the artificial intelligence community to adopt Texas Hold’em, the most popular variant, as a benchmark of progressively increasing difficulty: first the limit (fixed-bet) heads-up form, then no-limit heads-up, and finally no-limit multiplayer games against several simultaneous opponents.

The scientific arc of this program is remarkably complete. Early systems of the 1990s and 2000s relied on hand-crafted expert knowledge and opponent modeling [1]. The arrival of *counterfactual regret minimization* (CFR) in 2007 provided the first practical algorithm for approximating Nash equilibria in enormous imperfect-information games [23], and by 2015 the Cepheus project had essentially solved heads-up limit Hold’em [2]. In 2017 two independent systems, DeepStack [12] and Libratus [3], defeated human professionals in heads-up no-limit Hold’em—the game tree of which contains roughly 10^{160} decision points [15]. Two years later Pluribus extended superhuman play to six-player no-limit Hold’em [5], a setting where the very notion of a unique optimal strategy dissolves and agents must instead target robust, unexploitable policies.

This paper is written for readers who want both the mathematics and the algorithms in one place, at full rigor but without unexplained leaps. Three goals organise the presentation. First, we build the probabilistic core that any strong player or program must internalize: hand combinatorics (Section 3), equity and pot odds, expected value, and the variance that governs bankroll requirements. Second, we develop the algorithmic ladder in order of conceptual depth: rule-based expert systems and Monte Carlo estimation (Section 4), the extensive-form game-theoretic framework (Section 5), regret minimization and CFR with its modern variants (Section 6), and reinforcement learning in imperfect-information settings, including the architectures of DeepStack, Libratus, and Pluribus (Section 7). Third, we ground the entire discussion in a working implementation: a strictly rule-compliant No-Limit Hold’em engine with five AI paradigms at one table, which we use in Section 8 to quantify—via a 100,000-hand simulation—how abstract theory translates into observable differences in win rate and playing style.

The paper is deliberately self-contained. Every algorithm described is either derived from first principles or accompanied by pseudocode faithful to a working implementation; every probabilistic claim is either proved or reducible to a one-line application of a stated theorem. Where a rigorous treatment would require measure-theoretic machinery—for instance, the full generality of stochastic games—we restrict attention to the finite structures that Hold’em actually presents, which suffice for every result we state. Readers interested primarily in the practical study can jump directly to Section 8 after Sections 5 and 6, on which the experimental agents’ designs rest.

2 The Game of Texas Hold’em

2.1 Structure and terminology

Texas Hold’em is played with a standard 52-card deck. Each player receives two private *hole cards*; five *community cards* are dealt face up in stages and shared by all players. A hand consists of four betting rounds interleaved with the card deals: *preflop* (hole cards dealt), *flop* (three community cards), *turn* (one card), and *river* (one final card). At the end of each hand,

remaining players form their best five-card poker hand from the seven available cards (two hole plus five community); the player with the highest-ranked hand wins the pot, or the pot is split among tied players.

In the no-limit variant, at each decision a player may fold (surrender the hand and any chips already wagered), check or call (match the current wager, or pass if no wager is pending), or raise to any amount between a minimum raise size and the player’s entire remaining stack. Two forced wagers seed the pot: the *small blind* and *big blind*, posted before the cards are dealt by the two seats to the left of a *dealer button* that rotates one seat each hand. The minimum raise equals the size of the last raise or bet; a raise of less than the minimum is permitted only when it exhausts a player’s stack (a *short all-in*), in which case betting is not reopened for players who have already acted.

Figure 1 summarises the flow of a hand. Two details of correct rule enforcement deserve emphasis because naive implementations routinely botch them. First, when players are all-in and no further betting is possible, the remaining community cards are dealt and the hand proceeds directly to showdown, frequently producing layered *side pots*: a player can only win a share of the pot proportional to the amount they actually matched from each opponent. Second, the preflop round is closed by an option for the big blind: if every other player has merely called, the big blind may still raise, because posting the blind is not a voluntary action.

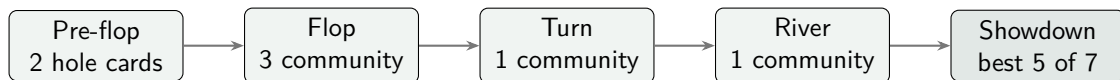


Figure 1: The four betting streets of a Texas Hold’em hand. Each street ends when all active players have matched the largest wager or are all-in; a hand may terminate prematurely by folds at any point, in which case the last remaining player takes the pot without showdown.

2.2 Hand rankings

Five-card hands are ranked in nine categories, each compared within itself by kickers: straight flush (five consecutive cards of one suit, an ace-high straight flush being the *royal flush*), four of a kind, full house (three of a kind plus a pair), flush, straight, three of a kind, two pair, one pair, and high card. The ranking is not arbitrary: as Section 3 quantifies, it reflects the number of card combinations that realise each category, so that rarer categories beat common ones with near-total reliability across the opponent’s distribution of holdings.

2.3 From cards to decisions

A single hand of Hold’em presents each player with at most four decision points (one per street, plus extra decisions for each raise cycle). Yet the state space is astronomical. Heads-up no-limit Hold’em has on the order of 10^{160} decision points when stack-to-pot ratios allow deep betting trees [15], and even the limit variant contains 10^{18} game states. Consequently, every practical algorithm in this paper couples a search, estimation, or learning procedure with an *abstraction* that compresses the state space to a tractable size. The art of poker AI is largely the art of designing abstractions whose equilibria are close to equilibria of the real game—a theme developed quantitatively in Sections 6 and 8.

3 Probability Foundations

3.1 Combinatorics of the deck

All card probabilities flow from one primitive: the number of ways to choose k unordered cards from n distinct cards,

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad (1)$$

because every hand in poker is an unordered subset of the deck, and (before any cards are observed) all subsets are equiprobable. Texas Hold'em uses the $k = 2$ and $k = 5$ cases at the moment cards are assigned to players, and the $k = 7$ case at showdown, where the player's effective hand is the *best five-card subset* of the seven available cards.

The number of distinct starting hands is

$$\binom{52}{2} = \frac{52 \cdot 51}{2} = 1326. \quad (2)$$

Suits are strategically interchangeable, so many holdings are equivalent up to suit isomorphism: there are 13 pocket pairs, $\binom{13}{2} = 78$ suited combinations and 78 offsuit non-pair combinations, giving 169 *canonical* starting hands. A specific pair such as $Q\spadesuit Q\heartsuit$ has probability $6/1326$ (six suit pairings), a specific suited hand such as $A\spadesuit K\spadesuit$ has probability $4/1326$, and a specific offsuit hand such as $A\spadesuit K\heartsuit$ has probability $12/1326$. These are the weights one must use when aggregating over the canonical classes.

3.2 Hand-category frequencies

Table 1 lists the exact number of five-card and seven-card combinations falling into each ranking category, together with the resulting probabilities. The five-card column is a classical computation; we illustrate with two categories and note that the rest follow the same bookkeeping. For a flush, choose the suit (4 ways), then choose 5 of that suit's 13 cards, and subtract the 10 straight flushes:

$$4 \binom{13}{5} - 40 = 5108. \quad (3)$$

For exactly one pair, choose the paired rank (13), the pair's suits $\binom{4}{2} = 6$, the three distinct kicker ranks from the remaining 12 ranks $\binom{12}{3}$, and one suit for each kicker (4^3):

$$13 \binom{4}{2} \binom{12}{3} 4^3 = 1098240. \quad (4)$$

The seven-card column counts the best five-card category of each of the $\binom{52}{7} = 133,784,560$ combinations. The inversions relative to the five-card distribution are striking and consequential. Among seven cards, one pair becomes the *modal* category (about 43.8%), while a no-pair high card collapses to 17.4%; a player holding five visible community cards plus two hole cards therefore makes at least a pair most of the time. Conversely, straights, flushes, and full houses all become far more common than their five-card frequencies suggest. Figure 2 visualises the distribution on a logarithmic scale spanning five orders of magnitude.

3.3 Preflop equity

The single most useful summary of a starting hand is its *equity*: the probability of winning (with ties counted fractionally) if all remaining cards were dealt and all hands revealed. Equity against a single uniformly random opponent can be computed exactly by enumeration or estimated by simulation; Figure 3 shows the values for all 169 canonical hands. The structure is instructive.

Table 1: Exact hand-category frequencies for five-card and seven-card poker hands. Probabilities are relative to $\binom{52}{5} = 2,598,960$ and $\binom{52}{7} = 133,784,560$ combinations respectively. The seven-card column reflects the *best* five-card hand available within each combination.

Category	5 cards		7 cards	
	Count	Probability	Count	Probability
Straight flush	40	0.00154%	41,584	0.0311%
Four of a kind	624	0.0240%	224,848	0.168%
Full house	3,744	0.144%	3,473,184	2.60%
Flush	5,108	0.197%	4,047,644	3.03%
Straight	10,200	0.392%	6,180,020	4.62%
Three of a kind	54,912	2.11%	6,461,620	4.83%
Two pair	123,552	4.75%	31,433,400	23.5%
One pair	1,098,240	42.3%	58,627,800	43.8%
High card	1,302,540	50.1%	23,294,460	17.4%

Pocket aces hold about 85% equity; the worst hand, $7\clubsuit 2\spadesuit$, holds about 35%—still far from hopeless heads-up, because even trash occasionally pairs or makes a backdoor straight. Suitedness is worth roughly 2–3 equity points for high-card hands (flush potential), and connectedness adds a fraction of a point (straight potential). The same grid, computed against multiple simultaneous random opponents, compresses dramatically: against five random hands, AA retains roughly 49% equity while 72o falls under 5%, which is why multiway pots systematically reward speculative hands and punish dominated high-card holdings.

3.4 Conditional probability and reads

Once the flop arrives, hand strength must be conditioned on the visible cards. If a player holds a flush draw—four cards of one suit—then among the 47 unseen cards, 9 complete the flush, so by a direct application of the definition of conditional probability,

$$\Pr(\text{flush by river} \mid \text{flop}) = 1 - \frac{\binom{47}{2} - \binom{9}{1}\binom{38}{1} - \binom{9}{2}}{\binom{47}{2}} = 1 - \frac{\binom{38}{2}}{\binom{47}{2}} \approx 0.3497. \quad (5)$$

Poker players memorise the practical rule that follows from (5): with two cards to come, multiply the number of *outs* by 4; with one card to come, by 2. The rule is accurate to within a couple of percentage points for the relevant range $1 \leq \text{outs} \leq 21$ precisely because it is the first-order binomial approximation of (5).

Bayes’ theorem is the formal home of *reading* an opponent. Write H for a hypothesis about the opponent’s holding (for instance, “at least one club”) and E for observed evidence (for instance, “the opponent called a large bet on a two-club board”). Then

$$\Pr(H \mid E) = \frac{\Pr(E \mid H) \Pr(H)}{\Pr(E)}, \quad \Pr(E) = \sum_{H'} \Pr(E \mid H') \Pr(H'). \quad (6)$$

Strong play requires an a priori model $\Pr(H)$ of the opponent’s distribution (a *range*) over holdings, and a behavioural likelihood $\Pr(E \mid H)$ describing how the opponent bets with each holding. Expert “table reading” is exactly the disciplined application of (6), updating the range after every action; opponent-modelling systems such as those reviewed by Billings et al. [1] mechanise the same computation.

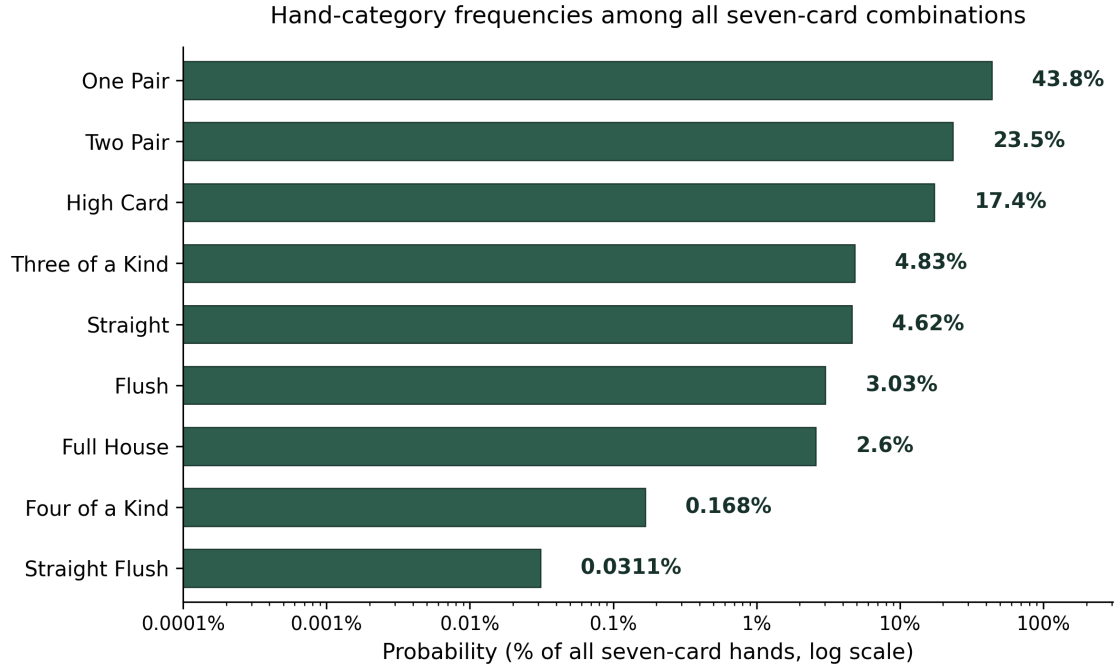


Figure 2: Frequency of each hand category among all $\binom{52}{7}$ seven-card combinations, on a logarithmic probability scale (categories ordered by seven-card frequency). Labels give percentages; the corresponding exact counts appear in Table 1.

3.5 Pot odds, equity, and expected value

Suppose the pot currently contains P chips and a player faces a bet of b chips. Calling costs b to contest a pot of $P + b$, so the call is profitable in expectation against the bettor's distribution whenever the caller's equity e satisfies

$$e > \frac{b}{P + b}. \quad (7)$$

The right-hand side is called the *pot odds* expressed as a required equity fraction. The inequality follows from a one-line expected-value computation: letting W be the indicator of winning the pot outright (ignoring ties),

$$\mathbb{E}[\text{profit}] = eP - (1 - e)b = e(P + b) - b, \quad (8)$$

which is positive exactly when (7) holds. As a concrete example: facing a half-pot bet of $b = P/2$, the threshold is $P/2 \div 3P/2 = 1/3$; a player with a nine-out flush draw on the turn ($9/46 \approx 0.196$ from (5) with one card to come) does not have direct odds and must either fold or justify the call through *implied odds*—expected additional winnings I on later streets when the draw completes, replacing (7) by

$$e > \frac{b}{P + b + I}. \quad (9)$$

Expected value is the fundamental quantity of the entire subject. If a strategy profile induces a distribution p over terminal outcomes with payoffs $v(z)$, the value of the strategy is

$$v = \mathbb{E}_p[v] = \sum_z p(z) v(z), \quad \text{Var}[v] = \sum_z p(z) (v(z) - v)^2. \quad (10)$$

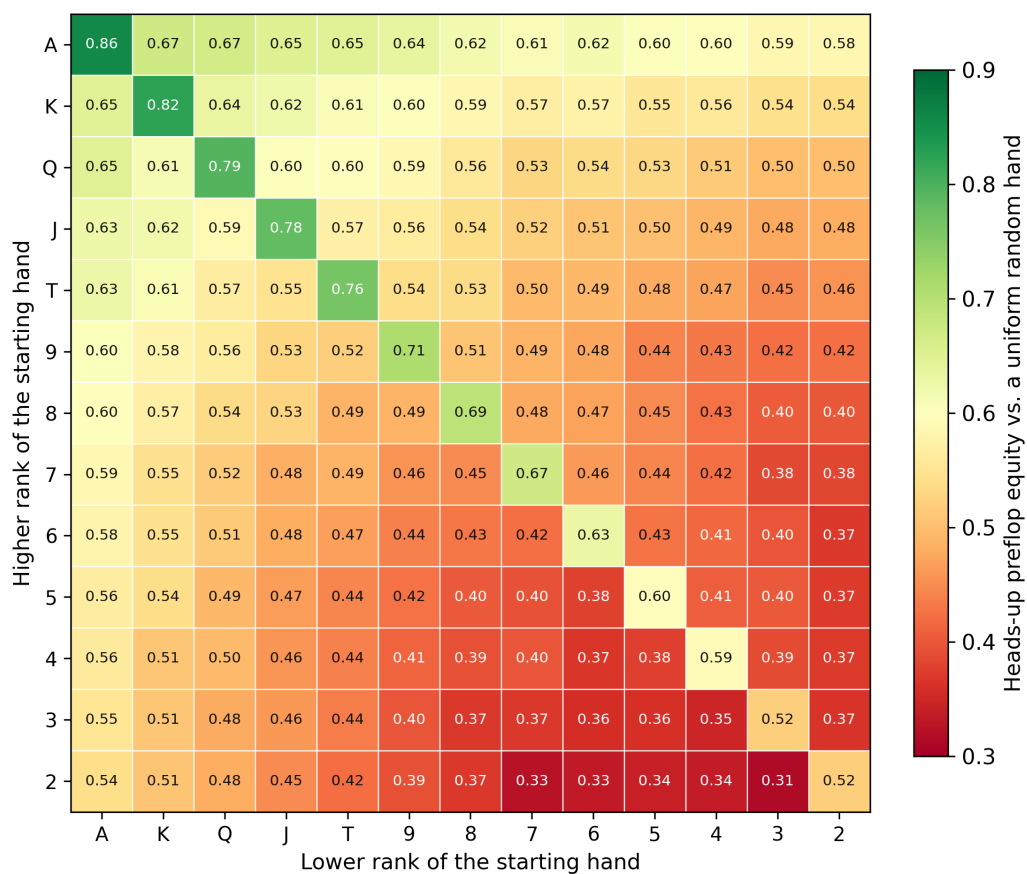


Figure 3: Heads-up preflop equity of all 169 canonical starting hands against a uniformly random opponent, estimated by Monte Carlo simulation with 4,000 runouts per hand. Values above the diagonal are suited hands; those below are offsuit.

Maximising (10) is the correct objective only when the distribution p is known or estimated; much of Sections 5–7 is devoted to the harder case where p is shaped by strategic opponents who hide information.

3.6 Variance, swings, and the Kelly criterion

Poker outcomes are dominated by variance. In a typical no-limit game, the per-hand profit of a solid player has standard deviation on the order of 10–15 big blinds (bb) while the expected profit is a few tenths of a bb; the ratio of noise to signal is therefore 20–50 per hand. It follows from the central limit theorem that after n hands the cumulative profit is approximately normal with mean $n\mu$ and standard deviation $\sqrt{n}\sigma$, so the number of hands needed to be 95% confident of a positive cumulative result is

$$n \geq \left(\frac{1.645\sigma}{\mu} \right)^2. \quad (11)$$

With $\sigma = 12$ bb and $\mu = 0.3$ bb this is roughly 43,000 hands—a first quantitative explanation of why judging poker skill from a single session is statistically meaningless, and why the simulation study of Section 8 requires tens of thousands of hands to separate its agents.

Bankroll management is the repeated-investment version of the same computation. If a player wins b units per unit staked with probability $p > 1/2$, Kelly’s classical result [9] states that the fraction of the bankroll to risk that maximises the asymptotic growth rate is

$$f^* = \frac{bp - (1 - p)}{b} = p - \frac{1 - p}{b}, \quad (12)$$

obtained by maximising $\mathbb{E}[\log(1 + fX)]$ over the stake fraction f . Because the log objective penalises ruin superlinearly, the Kelly fraction is conservative relative to the naive μ/σ^2 sizing; serious players typically adopt a fraction of Kelly between one quarter and one half to buffer estimation error in p and b . The deeper lesson transfers directly to the algorithms of Section 7: any learning or search procedure whose per-decision stakes are sized by noisy estimates must temper its aggression in proportion to estimation uncertainty.

4 Hand Evaluation, Monte Carlo, and Expert Systems

Before any strategic reasoning can occur, a program must answer two primitive questions quickly and exactly: *which five-card hand does a player hold* (ranking), and *how strong is a holding against a specified field of opponents* (equity). This section develops both primitives and then assembles them into the first rung of the algorithmic ladder: a rule-based expert system and a Monte Carlo simulation agent, the two paradigms that dominated computer poker before game-theoretic methods arrived.

4.1 Exact hand evaluation

A Hold’em evaluator is a function from seven cards to a totally ordered score. The natural construction exploits the fact that a player’s effective hand is the *best* of the $\binom{7}{5} = 21$ five-card subsets:

$$\text{score}(c_1, \dots, c_7) = \max_{S \in \binom{[7]}{5}} \text{rank}_5(S), \quad (13)$$

where rank_5 maps a five-card hand to a lexicographic tuple (category, primary kickers, ..., final kicker). Because the nine categories of Section 2.2 are mutually exclusive at fixed card multiplicity, and within-category comparison reduces to comparing kickers in order, rank_5 takes values in a set of 7,462 distinct equivalence classes, which pack into a single machine integer. Our implementation encodes (category, $k_1, k_2, \dots$) as base-15 digits, so that (13) becomes 21 integer

maximisations—about 1.5 ns per evaluation on commodity hardware. Production evaluators go further: the widely used two-plus-two table-driven evaluator precomputes multi-card lookup tables over the card space and evaluates seven cards in seven $O(1)$ table probes. The exact method matters little for correctness—all sound evaluators induce the same total order—but matters greatly for the training loops of Sections 6 and 7, whose wall-clock budgets are consumed almost entirely by evaluation.

4.2 Monte Carlo equity estimation

Exact ranking solves showdowns, but decisions require *equity*: the win probability of a holding against a distribution of opponent holdings and future boards. For a single uniformly random opponent this expectation has a closed form only in trivial cases; the standard tool is Monte Carlo estimation. Fix the player’s hole cards h and the visible board B , let U denote the unseen cards, and let e be the true equity. A *runout* samples the opponent’s hole cards $o \sim U$ and the board completion $r \sim U \setminus \{o\}$ uniformly without replacement; the estimator draws n i.i.d. runouts and returns

$$\hat{e}_n = \frac{1}{n} \sum_{i=1}^n W_i, \quad W_i = \mathbf{1}\{\text{score}(h \cup r_i) > \text{score}(o_i \cup r_i)\} + \frac{1}{m_i+1} \mathbf{1}\{\text{tie against } m_i \text{ opponents}\}, \quad (14)$$

where o_i is the sampled opponent holding, r_i the sampled board completion, and a tie splits the pot among the $m_i + 1$ tied hands.

Proposition 4.1. *$\hat{e}_n$ is an unbiased estimator of e with $\text{Var}(\hat{e}_n) = e(1 - e)/n$. Consequently $\text{SE}(\hat{e}_n) = \sqrt{e(1 - e)/n} \leq \frac{1}{2\sqrt{n}}$, and by the de Moivre–Laplace central limit theorem an asymptotic 95% confidence interval is $\hat{e}_n \pm 1.96 \sqrt{\hat{e}_n(1 - \hat{e}_n)/n}$.*

Proof. Each W_i is an independent draw of the random variable $W = \mathbf{1}\{\text{win}\} + (\text{fractional tie})$, whose mean is e by construction of the uniform sampling measure; ties contribute fractional mass so that $\mathbb{E}[W]$ equals the exact split-pot equity. The variance of a mean of n i.i.d. bounded variables in $[0, 1]$ with success probability e is $e(1 - e)/n$, maximised at $e = 1/2$. $\square$

The $1/\sqrt{n}$ decay is the governing constraint on every sampling-based poker agent: doubling precision quadruples the simulation budget. Figure 4 plots the standard error (4.1) against n and marks the two budgets used by our implementation: $n = 300$ for batch (tournament) play, giving $\text{SE} \approx 2.9$ equity points, and $n = 1500$ for interactive play with a 220 ms time budget, giving $\text{SE} \approx 1.3$ points. The practical reading is sobering: even 1,500 runouts cannot distinguish a 52% equity from a 49% equity at 95% confidence, yet such margins decide most real bets. Variance-reduction techniques (antithetic pairing of high/low runouts, stratified sampling of opponent hands by canonical class) can shave constant factors, but the square-root wall is information-theoretic: an agent that wants sharper beliefs must either enumerate (exactly, at combinatorial cost) or *condition on fewer unknowns*—which is precisely what the abstraction machinery of Section 6 achieves by design.

4.3 Rule-based expert systems

The oldest and still most common commercial paradigm encodes human expert knowledge as a decision procedure over computed statistics. Our implementation, which we call the *rule agent*, is a faithful miniature of this school. At each decision point it computes three quantities: a *strength* estimate $s \in [0, 1]$ —preflop, a table lookup of the 169 canonical hands’ Monte Carlo equities (Figure 3); postflop, a made-hand percentile with additive draw bonuses—a *position class* from $\{\text{EP}, \text{MP}, \text{LP}, \text{SB}, \text{BB}\}$, and the *pot-odds threshold* $b/(P + b)$ of (7). Decision rules

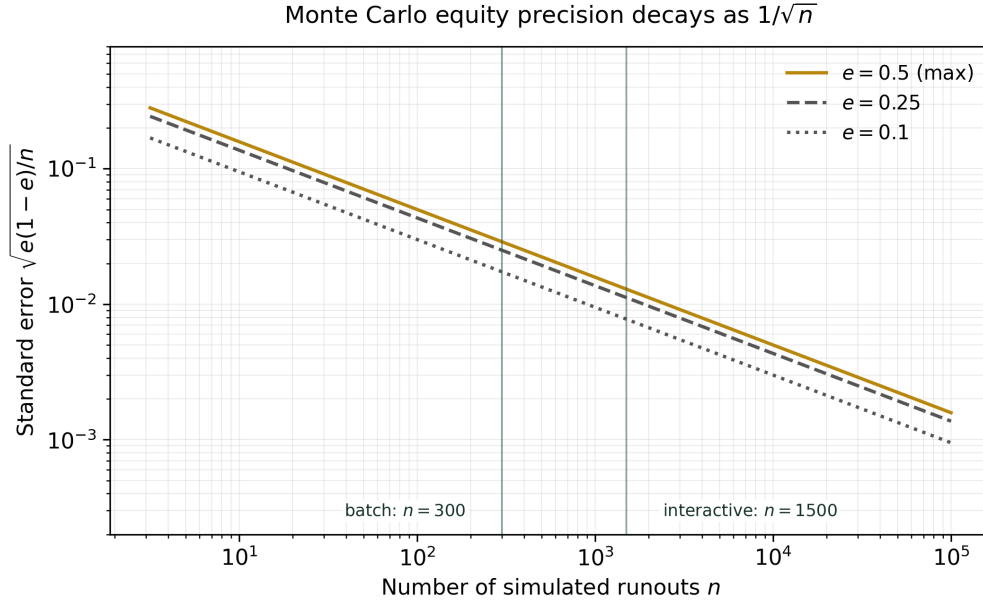


Figure 4: Standard error of the Monte Carlo equity estimator (14) as a function of the number of simulated runouts, for true equities $e \in \{0.5, 0.25, 0.1\}$. The vertical lines mark the two simulation budgets of our Monte Carlo agent: the batch budget $n = 300$ (worst-case SE ≈ 2.9 percentage points) and the interactive budget $n = 1500$ (SE ≈ 1.3 points).

then combine them: open-raise when s clears a position-dependent threshold (tighter in early position, looser on the button), three-bet premium holdings, call when s exceeds the required equity plus a multiway safety margin $0.03 + 0.025 \min(3, \text{opponents})$, value-bet strong made hands, and bluff at a low fixed frequency when last to act. Algorithm 1 gives the complete decision procedure.

The strengths of this paradigm are transparency (every action is explainable in terms a human recognises—the agent’s user interface displays exactly these threshold comparisons), speed (microseconds per decision), and robustness against catastrophic error. Its weaknesses are equally structural. First, the thresholds are *static*: the agent cannot adapt to an opponent who folds too much or calls too much, whereas a best responder profits precisely from such regularities. Second, and more fundamentally, the rule agent has no theory of *balanced ranges*: because its aggression depends only on its own card strength, its betting ranges are transparently polarised or merged, and its bluff frequency is an arbitrary constant rather than a quantity chosen to make the opponent indifferent. A game-theoretic opponent can therefore exploit it systematically, a defect that Section 8 quantifies in win-rate terms. The Monte Carlo agent improves the *estimation* half of the problem—it replaces the heuristic strength s with a live simulation against the actual number of opponents, with an honest confidence interval—but inherits the same static decision thresholds, so its fate against adaptive opponents is similar.

Both paradigms share a deeper limitation: they treat each decision in isolation, as a myopic expected-value calculation against a *fixed* model of the opponents. Poker’s central difficulty is that the model is not fixed—opponents’ strategies respond to ours, and their private cards generate adversarial uncertainty that no amount of sampling removes. Formalising this observation requires the language of game theory, to which we now turn.

Algorithm 1: Rule-based decision at one betting node (our *rule agent*).

Input: state (hole, board, pot P , to-call b , position ρ , opponents m), random draw $r \sim U(0, 1)$
Output: action $\in \{\text{fold, check/call, raise } \frac{1}{2}\text{-pot, raise pot}\}$

```
1  $s \leftarrow$  preflop-equity lookup or made-hand percentile + draw bonuses
2 if  $b = 0$  then
3   if  $s \geq \theta_{\text{open}}(\rho)$  then
4     if  $s \geq 0.62$  and  $r < 0.35$  then
5       emit pot-sized raise
6     else
7       emit half-pot raise
8   else
9     check (fold-option hands check behind)
10  else
11     $\theta \leftarrow b/(P + b) + 0.03 + 0.025 \min(3, m) - \mathbb{I}[\rho \in \{\text{SB, BB}\}] \cdot 0.02$ 
12    if  $s \geq 0.68$  and raises this street  $< 3$  and  $r < 0.55$  then
13      emit 3-bet
14    if  $s \geq \theta$  then
15      call
16    if  $\theta < 0.15$  and  $s \geq 0.42$  then
17      call (cheap peeling / set-mining)
18    return fold
```

5 Game-Theoretic Foundations

5.1 Extensive-form games with imperfect information

The correct formal model for poker is a finite extensive-form game with imperfect information. It consists of:

1. a finite game tree rooted at h_0 ; internal nodes h belong to a player-mapping $P : H \rightarrow N \cup \{c\}$, where $N = \{1, \dots, n\}$ are the players and c is the chance player;
2. chance nodes prescribe probability $\sigma_c(h, a)$ to each child $h \cdot a$ (in Hold'em, uniform over the unseen cards);
3. payoffs $u_i : Z \rightarrow \mathbb{R}$ at terminal nodes $z \in Z$ (in Hold'em, chip deltas);
4. for each player i , a partition $\mathcal{I}_i$ of that player's decision nodes into *information sets* I : nodes in one I are indistinguishable to i because they differ only in information i cannot observe (opponents' cards).

A player's memory of its own past observations is encoded by the partition: the game has *perfect recall* if, whenever two nodes h, h' lie in the same information set of player i , the sequence of i 's information sets and actions leading to h and to h' is identical—informally, the player never forgets what it once knew or did. All poker variants have perfect recall, which matters because of Kuhn's theorem: with perfect recall, behavioural strategies (independent action distributions at each information set) are payoff-equivalent to mixed strategies (distributions over deterministic plans), and the two representations can be interchanged freely [10]. Every algorithm in this paper therefore operates on behavioural strategies σ_i , which assign to each $I \in \mathcal{I}_i$ a distribution over the actions legal at I .

The number of information sets is the true measure of a poker game's size. Heads-up limit Hold'em has about 10^{17} of them; heads-up no-limit, on the order of 10^{160} counting distinct bet

sizes; six-player no-limit is larger still [15]. Any tabular method that touches each information set once is hopeless at this scale—a fact that drives every abstraction decision in Section 6.

5.2 Nash equilibrium and its guarantees

A strategy profile $\sigma = (\sigma_1, \dots, \sigma_n)$ is a *Nash equilibrium* if no player gains by unilateral deviation: for every i and every alternative σ'_i ,

$$u_i(\sigma'_i, \sigma_{-i}) \leq u_i(\sigma). \quad (15)$$

Nash’s theorem guarantees that every finite game has at least one such profile (in mixed strategies; with perfect recall, in behavioural strategies) [13]. The concept’s computational meaning for two-player *zero-sum* games is sharper. Define a player’s *best response* value $\text{BR}_i(\sigma_{-i}) = \max_{\sigma'_i} u_i(\sigma'_i, \sigma_{-i})$ and the *exploitability* of a profile by

$$\varepsilon(\sigma) = \frac{1}{2}[\text{BR}_1(\sigma_2) - \text{BR}_2(\sigma_1)] = \frac{1}{2} \left[\max_{\sigma'_1} u_1(\sigma'_1, \sigma_2) - \min_{\sigma'_2} u_1(\sigma_1, \sigma'_2) \right], \quad (16)$$

which vanishes exactly at equilibrium. Two consequences make equilibrium play the reference standard for competitive poker AI. First, by von Neumann’s minimax theorem the zero-sum game has a *value* v^* : any equilibrium profile attains it, player 1 can guarantee at least v^* against *any* opponent, and player 2 can hold any opponent to at most v^* . A maximally unexploitable strategy is therefore also a maximally safe one. Second, approximate equilibria inherit approximate safety: if $\varepsilon(\sigma) \leq \epsilon$, then a best response against σ earns at most $v^* + \epsilon$, *regardless of the exploiter’s knowledge or adaptivity*. This is the formal licence to search for ϵ -equilibria of abstracted games and treat the result as robust—the licence under which Cepheus, Libratus, and Pluribus were built [2, 3, 5].

Beyond two players these guarantees dissolve. With three or more players the game is no longer zero-sum (one player’s loss is not another’s gain), equilibrium values are not interchangeable, and some equilibria are collusively bad for individual players: in a six-player game, a strategy that is a best response to one profile of opponents can lose heavily against a different profile of the *same strength*. Equilibrium strategy in multiplayer poker is therefore a *robustness heuristic*—a guarantee against being the table’s designated victim—rather than an optimality certificate. This is exactly how Pluribus was evaluated and is why the six-agent study of Section 8 reports pairwise-style aggregate statistics rather than exploitability [5].

5.3 Why perfect-information methods fail

Backward induction on a game tree requires evaluating nodes; a node’s value is well-defined only when the evaluator can condition on everything the mover knows. In poker the mover’s knowledge is coarser than the tree: two nodes in one information set demand one action distribution yet differ in continuation values, because the hidden cards alter every downstream payoff. Any procedure that solves subtrees independently—alpha-beta search, expectimax, dynamic programming over the tree—silently conditions on the hidden state and thereby assumes the mover can read opponents’ cards. The result is a strategy whose apparent value is an unreachable upper bound, and whose realised value against a live opponent collapses: a “best strategy against known cards” bluffs never and folds to every raise.

The failure has a constructive side: the correct equilibrium behaviour in imperfect-information games is generally *mixed*. The trivial game rock-paper-scissors (a one-move simultaneous game) makes the point irreducibly: any deterministic strategy is exploited with certainty, and the unique equilibrium randomises uniformly over the three actions. Poker differs only in that the mixing probabilities are non-obvious, card-dependent, and coupled across a vast tree. The next example shows that even in the smallest poker game the mixing probabilities obey exact arithmetic that no intuition produces unaided.

5.4 A complete worked example: Kuhn poker

Kuhn poker is the smallest game recognisably belonging to the poker family [10]. The deck has three cards $J < Q < K$; each player antes one unit and receives one card; bet size is fixed at one unit. Player 1 (P1) acts first and may check or bet; if P1 bets, P2 calls or folds; if P1 checks, P2 checks (showdown) or bets, whereupon P1 calls or folds. The game has 12 information sets—six per player (one per card at the first decision, one per card at the second)—and can be solved exactly.

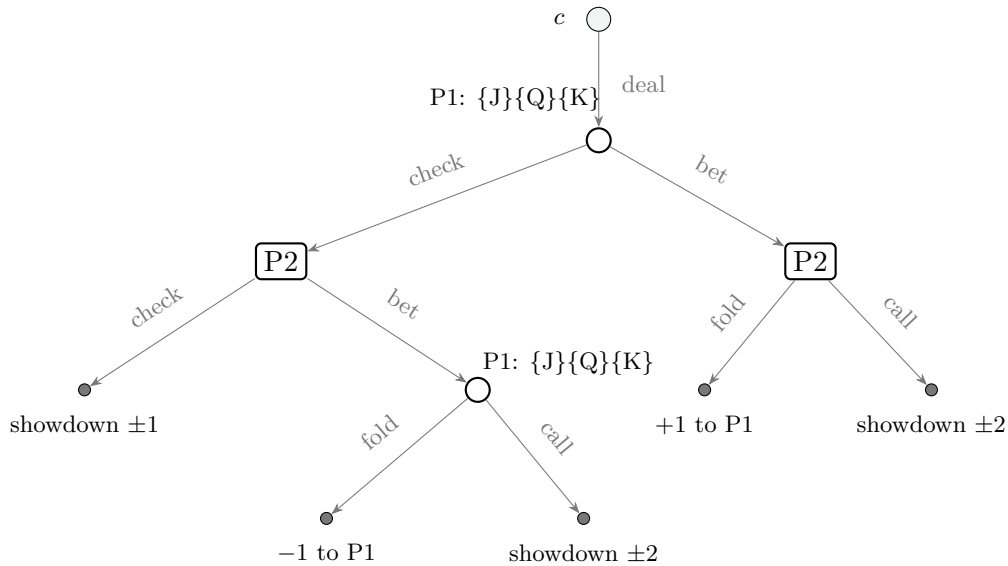


Figure 5: The Kuhn poker tree (card-deal branches and per-card information sets suppressed; each decision node shown aggregates three information sets, one per private card). Both players see only their own card, so every decision aggregates three indistinguishable deals.

Proposition 5.1 (Kuhn [10]). *The value of Kuhn poker to Player 1 is $v^* = -1/18$. Moreover the equilibria form a one-parameter family: for every bluffing rate $\alpha \in (0, \frac{1}{3}]$ the following profile is an equilibrium, and every equilibrium restricts to this form:*

- **P1:** bet the jack (bluff) with probability α ; bet the king with probability 3α ; always check the queen; facing a bet after checking, fold the jack, call with the queen at the indifference rate, always call with the king;
- **P2:** facing a bet, fold the jack, call with the queen with probability $\frac{1}{3}$, always call with the king; after a check, bluff the jack with probability $\frac{1}{3}$, always check the queen, always bet the king.

Three features deserve emphasis, because they scale unchanged to full poker. First, *the value is negative for the first player*: acting first is a disadvantage worth $1/18 \approx 5.6\%$ of the ante per hand, a positional tax that persists (in larger magnitude) in Hold'em and explains why seat selection matters. Second, *bluffing is mandatory and exactly priced*: P1 bluffs the worst hand at rate α but must bet the best hand at rate 3α , so that a bet is a bluff exactly one quarter of the time—precisely the pot odds P2 faces when calling one unit into a pot of three ($b/(P+b) = 1/4$ in (7)). At this ratio, and only at this ratio, P2's queen is indifferent between calling and folding; if P1 bluffs more, calling profits, and if less, folding profits. The equilibrium mixing probabilities are not free parameters of psychology but *arithmetic consequences of the pot odds*, a fact that

generalises to every no-limit bet size. Third, *trapping emerges*: P1 never bets the queen, the middle hand, because betting it would populate the betting range with a hand that loses to K and dominates nothing, corrupting the 1:3 bluff-value ratio; instead the queen checks and collects calls from bluffs. In Hold'em this pattern reappears as the modern “polarised betting range”—bet the nuts and the worst bluffs, check the middling hands.

We verified Proposition 5.1 computationally: a self-contained vanilla CFR implementation (Section 6) run for 2×10^5 iterations converges to a profile whose exact value is -0.055556 (to six decimals, $= -1/18$) and whose exploitability (16), computed by brute-force enumeration of all 2^{12} pure profiles, is 3.6×10^{-4} of the ante. The converged strategy exhibits the family exactly: the measured bluff and value-bet rates were $\alpha = 0.233$ and $3\alpha = 0.698$, and P2’s two mixing frequencies were 0.334 and 0.335. This tiny game is the paper’s microscope for equilibrium structure; the remainder of the paper is devoted to making the same construction tractable at Hold'em’s scale.

6 Counterfactual Regret Minimization

Counterfactual regret minimization (CFR), introduced by Zinkevich et al. in 2007, is the algorithm that made large imperfect-information games tractable [23]. Its idea is to decompose a game’s overall regret—an intractably global quantity—into independent per-information-set regrets, each minimised by a simple adaptive rule, and to prove that minimising the parts minimises the whole. We develop CFR in three steps: the no-regret learning primitive (regret matching), the decomposition theorem, and the sampling and function-approximation variants used in practice, including our own implementation whose training run and tournament behaviour are reported in Section 8.

6.1 Regret and regret matching

Fix a repeated decision problem with action set A , and let the adversary choose a sequence of payoff vectors $v^1, v^2, \dots, v^T$ with entries in $[-\Delta, \Delta]$, where $v^t(a)$ is the payoff of playing action a at time t . An online learner produces a sequence of mixed strategies $\sigma^1, \dots, \sigma^T$ without seeing the future. Its *external regret* against the best fixed action in hindsight is

$$R^T = \max_{a^* \in A} \sum_{t=1}^T \left[v^t(a^*) - \sum_{a \in A} \sigma^t(a) v^t(a) \right]. \quad (17)$$

A learner is *no-regret* if $R^T/T \rightarrow 0$; the rate at which R^T grows in T is the learner’s quality.

Regret matching maintains cumulative per-action regrets $r^t(a)$ —the running total of a ’s payoff minus the realised payoff of the mixed play—and plays the positive part, normalised:

$$\sigma^{t+1}(a) = \frac{\max(0, r^t(a))}{\sum_{a'} \max(0, r^t(a'))}, \quad \text{with } \sigma^{t+1} \text{ uniform if the denominator is 0.} \quad (18)$$

Lemma 6.1. *Regret matching satisfies $R^T \leq \Delta \sqrt{(|A| - 1)T}$ for all T .*

Proof sketch. Use the potential $\Phi^t = \sum_a \max(0, r^t(a))^2$. A Blackwell-condition computation shows $\Phi^t - \Phi^{t-1} \leq \sum_a (v^t(a) - \bar{v}^t)^2 \leq \Delta^2(|A| - 1)$ at every step, and $R^T \leq \sqrt{|A| \Phi^T}$ relates the potential to the regret. Chaining the two bounds and dividing by T gives average regret $O(\Delta \sqrt{|A|/T})$; see [7] for the complete argument. $\square$

The beauty of (18) is what it does *not* require: no knowledge of the adversary, no lookahead, and yet the learner’s time-average strategy is essentially as good as the best fixed action in hindsight against the actual sequence of opponents encountered. In poker the adversary is the joint behaviour of all other players and the deal of the cards, which is exactly the adversarial uncertainty that Sections 4–5 could not confront.

6.2 The CFR decomposition

To lift regret matching from single decisions to game trees, define for player i , information set I , and action a the *counterfactual value*

$$v_i(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \in Z} \pi_{haz}^\sigma u_i(z), \quad (19)$$

where $\pi_{-i}^\sigma(h)$ is the product of all chance probabilities and opponents' action probabilities on the path to h (i 's own probabilities excluded—hence *counterfactual*: the value of I to i were i to play to reach it), and π_{haz}^σ is the probability of continuing from $h \cdot a$ to terminal z . The counterfactual regret of I is $r_i(I, a) = v_i(I, a) - \sum_{a'} \sigma_i(a'|I) v_i(I, a')$. The key theorem is a decomposition:

Theorem 6.2 (Zinkevich et al. [23]). *If each player i plays at every information set $I \in \mathcal{I}_i$ by regret matching (18) applied to its counterfactual regrets, then player i 's overall average regret in the full game obeys*

$$\bar{R}_i^T \leq \frac{\Delta_i}{T} \sum_{I \in \mathcal{I}_i} \sqrt{T|A(I)|} = \Delta_i \sum_{I \in \mathcal{I}_i} \sqrt{\frac{|A(I)|}{T}}, \quad (20)$$

where Δ_i is the range of i 's payoffs. In particular $\bar{R}_i^T \rightarrow 0$ at rate $O(1/\sqrt{T})$, and by the definition of regret the average strategy profile $\bar{\sigma}^T$ (the reach-weighted mean of the played σ^t) is a $(\sum_i \bar{R}_i^T)$ -Nash equilibrium in two-player zero-sum games, hence $\varepsilon(\bar{\sigma}^T) \leq \sum_i \bar{R}_i^T$.

Two remarks translate the theorem into practice. First, the bound is *additive over information sets*, so the algorithm never solves a global optimisation problem: each information set runs its own tiny regret matcher, and equilibrium emerges from the interaction. Second, the bound's dependence on $|\mathcal{I}_i|$ is the formal statement of “the game is too big”: for 10^{17} information sets, direct tabular CFR needs astronomically many iterations before (20) becomes meaningful—hence the abstraction and sampling machinery below. Algorithm 2 states vanilla CFR in full.

CFR⁺. Tammelin's CFR⁺ modifies (18) by clamping cumulative regrets at zero after every update, alternating the updating player between iterations, and weighting the average strategy linearly in t [19]. No new theory is required—the variant remains a no-regret procedure—but convergence in practice improves by more than an order of magnitude; CFR⁺ was the engine behind essentially solving heads-up limit Hold'em [2]. Our training pipeline adopts the clamped-regret alternation with uniform averaging, which retains the no-regret guarantee of the underlying scheme.

Monte Carlo CFR. Vanilla CFR touches every node of the tree per iteration; even in an abstracted game this is prohibitive. Monte Carlo CFR (MCCFR) samples the traversal so that each counterfactual regret increment is an *unbiased* estimator of its exact value, then scales the update by the inverse sampling probability [11]. The workhorse variant is *external sampling*: at each iteration one traverser i is fixed; the walk enumerates *all* of i 's actions at i 's nodes (computing each child's value exactly along the sampled line) while opponent actions and chance deals are sampled from their current strategies. Per-iteration cost drops from the full tree to a single sampled line times the traverser's branching factor, and the estimator variance concentrates on the opponent/chance realisations, which is precisely where poker's uncertainty lives. External-sampling MCCFR with CFR⁺ regrets is the training algorithm of both Libratus and Pluribus [3, 5]—and of the CFR agent in our implementation, which trains a six-player blueprint by self-play with a rotating dealer button and pooled average strategies across seats.

Algorithm 2: CFR (one iteration; alternating player updates, CFR⁺ variant).

Input: game tree, cumulative regrets $r(I, \cdot)$, cumulative strategies $s(I, \cdot)$, updating player i

```

1 function CFR ( $h, \pi_i, \pi_{-i}$ ):
2   if  $h \in Z$  then
3     return  $u_i(h)$ ;
4   end
5   if  $P(h) = c$  then
6     return  $\sum_a \sigma_c(h, a)$  CFR ( $ha, \pi_i, \pi_{-i} \sigma_c(h, a)$ );
7   end
8    $I \leftarrow$  information set containing  $h$ ;  $\sigma(I, \cdot) \leftarrow$  regret matching (18) on  $r(I, \cdot)$ ;
9   if  $P(h) = i$  then
10     $s(I, a) += \pi_i \sigma(I, a)$  for all  $a$ ; // average strategy accumulator
11    for each action  $a$  do
12       $v(a) \leftarrow$  CFR ( $ha, \pi_i \sigma(I, a), \pi_{-i}$ );
13    end
14     $v \leftarrow \sum_a \sigma(I, a) v(a)$ ;
15     $r(I, a) += \pi_{-i} (v(a) - v)$  for all  $a$ ; // CFR+:  $r \leftarrow \max(0, r)$  after update
16    return  $v$ ;
17  end
18  // opponent (or non-updating player): sample or enumerate  $\sigma(I, \cdot)$ 
19  return  $\sum_a \sigma(I, a)$  CFR ( $ha, \pi_i, \pi_{-i} \sigma(I, a)$ );
20 Output: after  $T$  iterations,  $\bar{\sigma}(I, \cdot) = s(I, \cdot) / \sum_a s(I, a)$  is the (approximate) equilibrium strategy.
```

6.3 Abstraction: making the game fit

CFR’s bound (20) is only useful if $\mathcal{I}$ is small. Abstraction shrinks the game in two orthogonal directions.

Card abstraction. Hands of similar strength are merged into *buckets*. Preflop, the 1,326 concrete holdings collapse—losslessly, by suit isomorphism—to 169 canonical classes, which our pipeline further compresses to 20 percentile buckets of preflop equity. Postflop, our implementation computes an approximate strength percentile (made-hand category with kicker interpolation, plus additive bonuses for flush draws and straight draws) and buckets it into 10 classes. Production systems cluster the $\binom{50}{2} = 1,225$ river hand classes by k -means on earth-mover distance between equity distributions, a construction due to Schnizlein et al. that preserves far more strategic structure [16].

Action abstraction. No-limit betting is continuous; trees are not. The standard fix is a small menu of abstract bet sizes: ours is $\{f, c, h, p\} = \text{fold, check/call, raise to roughly half the pot, raise to roughly the pot, with raises capped at three per street and a deterministic mapping from abstract char to a legal concrete raise (opening raises of } 2.5\times \text{ and } 4\times \text{ the big blind preflop, half-pot and pot postflop, re-raises to } 2\times \text{ and } 2.75\times \text{ the current wager, all clamped to legal minimums and to the stack)}$. Libratus went further and refined its bet-size abstraction *during* play by adding sizes its opponents preferred [3].

The information-set space then factorises as a product: position class (five values, button-relative), street (four), card bucket (20 preflop / 10 postflop), and a compressed betting pattern (counts of raises, calls, folds within the street). Trained by external-sampling MCCFR until the per-iteration rate stabilised, our blueprint covers 13,873 distinct information-set keys reached in self-play—down from the 10^{160} scale of the raw game by virtue of the abstraction, at the price of strategic fidelity.

The price of abstraction. An equilibrium of the abstract game is generally *not* an equilibrium of the full game, for two distinct reasons. Mapping different full-game states into one abstract state forces the abstract strategy to play them identically, which is optimal only if their equilibrium strategies truly coincide; and the abstract game contains bet sizes and sequences absent from the full game’s realised play, so off-tree evaluation mis-prices the translated strategy. The phenomenon is measurable—a best responder allowed to bet arbitrary sizes exploits blueprint strategies trained on coarse action abstraction, an effect documented by Lanctot et al. and countered in research systems by finer size ladders and by subgame re-solving at play time [21, 4]. In our experimental setting, all agents share one action abstraction, so abstraction error cancels from the *comparison* even though it remains part of each agent’s absolute distance from true equilibrium.

6.4 Deep CFR: regrets as a learned function

Tabular CFR stores one regret vector per information set, so the table’s size is the game’s size. Deep CFR replaces the tables with a neural network $\hat{v}_\theta : \mathcal{S} \rightarrow \mathbb{R}^{|A|}$ that *predicts* counterfactual advantages from state features [6]. During MCCFR traversals, (feature vector, advantage) pairs are reservoir-sampled into a replay buffer; the network is periodically trained by minimising squared error against the sampled advantages; and regret matching is applied to the *predicted* advantages at play time. The theoretical cost is explicit: if the network’s prediction error is bounded by δ in the appropriate weighted norm, the regret bound (20) degrades by an additive $O(\delta)$ term, so the learned strategy is an approximate equilibrium of the abstract game to the accuracy of the function approximator. The benefit is that $\hat{v}_\theta$ *generalises* to information sets the traversal never visited—the uniform-fallback failure mode of a sparse table disappears.

Our implementation is a deliberately miniature Deep CFR. The feature vector has 14 entries (street one-hot, normalised hand bucket, pot-odds fraction, pot-to-stack ratio, active-player fraction, position one-hot, can-raise indicator); the network is a $14 \rightarrow 32 \rightarrow 4$ multilayer perceptron with tanh hidden units trained by momentum stochastic gradient descent on a 60,000-sample reservoir. On held-out validation data the trained network predicts MCCFR advantages with a Pearson correlation of 0.44—honest numbers for a model of this size, and far below the fidelity of production systems, but sufficient to produce a playing style measurably distinct from its tabular sibling, as Section 8 quantifies. The miniature exists precisely to make the architecture’s moving parts—reservoir sampling, advantage regression, regret matching on predictions—inspectable end to end.

7 Reinforcement Learning in Imperfect-Information Games

The final rung of the algorithmic ladder replaces equilibrium computation with *trial and error*: an agent that learns by playing, observing rewards, and updating its own policy. Reinforcement learning (RL) is the most flexible of the paradigms surveyed here—it needs no game-tree enumeration, no explicit opponent model, and no hand-crafted thresholds—and consequently faces the sharpest theoretical hazards. This section sets out the classical RL framework, explains precisely where poker breaks its assumptions, describes the self-play designs that restored soundness (culminating in DeepStack, Libratus, and Pluribus), and specifies our own tabular Q-learning agent, which shares the CFR agents’ abstraction so that Section 8 can attribute performance differences to the learning paradigm rather than the state representation.

7.1 Markov decision processes and Q-learning

The classical substrate is the Markov decision process (MDP): states $s \in \mathcal{S}$, actions a , transition kernel $P(s' \mid s, a)$, rewards $r(s, a)$, and discount $\gamma \in [0, 1)$. The Bellman optimality equation

characterises the optimal action-value function:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\max_{a'} Q^*(s', a') \right]. \quad (21)$$

Tabular Q-learning iterates the stochastic approximation

$$Q(s_t, a_t) \leftarrow (1 - \alpha_t) Q(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right], \quad (22)$$

whose target $r_t + \gamma \max_{a'} Q(s_{t+1}, a')$ is itself a biased estimate built from the current Q —a *bootstrapped* update. Watkins’ theorem guarantees convergence $Q \rightarrow Q^*$ under standard stochastic-approximation conditions: every state–action pair visited infinitely often (sufficient exploration), and step sizes α_t decaying appropriately [22]. In a poker engine the episode structure is natural: one hand of play is an episode, the terminal reward is the chip delta, and (22) is propagated backwards over the seat’s decision sequence within the hand.

7.2 Where the substrate breaks

Poker violates every MDP assumption in turn. The state is not observed: the transition “kernel” depends on opponents’ hidden cards, so the learner’s information state is a belief, not a state, and the process over information states is not Markov in general. The environment is not stationary: opponents adapt, so the very dynamics being estimated drift as the learner improves—the condition $\alpha_t \rightarrow 0$ with infinite visitation becomes meaningless when the thing visited keeps changing. And the objective itself bifurcates: maximising (21) against a *fixed* environment is exploitation, whereas Section 5 established that adversarial settings reward unexploitability. An RL agent trained against one opponent pool learns to beat that pool and may thereby learn habits that a different opponent punishes; the analogous failure of the rule-based agent (Section 4) was static thresholds, and the cure—equilibrium reasoning—is the same.

The classical remedy is *self-play*: let the environment be other learners of the same family. Fictitious play, the grandfather of self-play schemes, has each player best-respond to the time-average of its opponents’ past play; Robinson proved convergence to equilibrium in two-player zero-sum games [14]. Self-play with function approximation underlies the landmark game-playing results—TD-Gammon’s surprise strength in backgammon [20] and the AlphaGo/AlphaZero line in perfect-information games [17]. In imperfect-information games, however, naive deep self-play can converge to brittle, mutually overfit policies, which motivated the hybrid architectures below.

NFSP. Neural fictitious play (NFSP) makes Robinson’s construction scalable: each agent carries *two* policies, a best response learned by deep Q-learning against the recent opponent distribution, and an average policy learned by supervised regression over the agent’s own history of best-response play [8]. Playing the average policy approximates fictitious play (hence equilibrium convergence in two-player zero-sum), while periodically best-responding supplies the exploitation pressure that keeps the average honest. NFSP was the first RL-based agent to approach competitive strength in heads-up limit Hold’em without explicit equilibrium computation, and its two-headed design—exploit *and* average—remains the template for reconciling RL with game theory.

The deadly triad. Wherever function approximation meets bootstrapping and off-policy data, value iteration can diverge—Sutton and Barto’s “deadly triad” [18]. Deep self-play adds a fourth hazard: the data distribution shifts as all policies move. Production mitigations (target networks, replay reservoirs, trust regions) are engineering responses to instability rather than fixes for it; our miniature deliberately stays tabular, accepting a small state space instead of an unstable one, and quantifies the cost of that choice in Section 8.

7.3 The superhuman architectures

DeepStack (2017). DeepStack’s designers rejected the blueprint-plus-search template and instead *continually re-solved* the remaining subgame at every decision [12]. A re-solve starts from the current belief over the opponent’s holdings (continual re-solving: the belief is inherited from the previous re-solve’s strategy, not recomputed), solves the depth-limited continuation of the game tree by CFR-style iteration, and evaluates *leaf* information sets with a deep network trained offline on millions of exactly solved subgames. The soundness argument is the paper’s theoretical contribution: because each leaf evaluation enters the solved value linearly, the strategy’s exploitability is bounded by the leaf-network’s error weighted by its influence—unlike heuristic forward search, the procedure degrades gracefully and provably. DeepStack’s second network evaluated full-hand distributions preflop, and its cascades of re-solves defeated a panel of human professionals in heads-up no-limit Hold’em.

Libratus (2017). Libratus is the archetype of the blueprint-plus-search school [3]. Its blueprint—an equilibrium of a coarse abstraction computed by external-sampling MCCFR with CFR⁺-style updates over 64-core months of self-play—played the early streets. From the third betting round onward, Libratus *re-solved* the actual subgame in real time on the unabridged betting sequence, using a nested safety condition to guarantee the re-solved strategy is no more exploitable than the blueprint’s recommendation. Between sessions, a *self-improver* tightened the endgame abstraction where its human opponents had concentrated their play (identified via their shown-down hole cards), so the system strengthened overnight against the specific humans it was fighting. Libratus won its 2017 bridge match against four human specialists by 1.76 million chips over 120,000 hands—a margin far beyond statistical doubt.

Pluribus (2019). Pluribus carried the template to six-player no-limit Hold’em, where equilibrium guarantees dissolve (Section 5.2) and compute budgets matter [5]. Its blueprint cost about \$150 of cloud compute—three orders of magnitude less than AlphaGo’s hardware—because it used a cheap “bubble” abstraction and a modest MCCFR budget. Live play used *limited-lookahead search*: within the current betting round, Pluribus enumerated unabridged betting sequences a few actions deep and evaluated leaf information sets with the blueprint’s continuation values, with a “sticky” variant of card abstraction to keep leaf evaluations consistent across the search. Against five humans at once, Pluribus won at rates professional players described as unsurprising for a top human specialist; the AI rating systems of the day considered it superhuman. The lesson Pluribus adds to Libratus is architectural parsimony: equilibrium-quality search, applied surgically at the decision points that matter, converts a modest blueprint into a superhuman whole.

7.4 Our RL agent: tabular Q-learning with an opponent pool

Our implementation instantiates the paradigm in its honest, small-scale form. The agent learns over *exactly the same information-set key space* as the CFR agents—position class, street, card bucket, betting pattern—so that the experimental comparison of Section 8 isolates the learning rule. Training is tabular Q-learning (22) in a six-seat self-play loop in which the learner occupies one seat and a *mixed opponent pool* occupies the other five: three rule agents, one uniform-random agent, and one CFR-blueprint agent. The pool’s composition is a deliberately cheap stand-in for NFSP’s averaging: facing a stationary mixture of heterogeneous opponents keeps the target of (22) approximately stationary while exposing the learner to value-oriented, erratic, and equilibrium-flavoured opposition in fixed proportion. Rewards are the terminal chip delta of each hand, discounted with $\gamma = 0.9$ and propagated backwards over the learner’s decision sequence; exploration is ϵ -greedy with ϵ annealed during training (and held at 0.02 in play).

No poker knowledge is hand-coded: aggression thresholds, positional discipline, and pot-odds awareness emerge, or fail to emerge, from the reward signal alone.

Three limitations should be stated plainly, because Section 8 measures their consequences. First, Q-learning here optimises *exploitation of the pool*, not equilibrium play: against the specific five-opponent mixture the learned policy is near-optimal, but nothing protects it outside that mixture—the guarantee asymmetry of Section 5.2 in miniature. Second, the tabular representation ties the agent to the shared abstraction, so it inherits the abstraction’s blind spots without Deep CFR’s ability to generalise across them. Third, the discounted-terminal-reward formulation teaches the agent hand-level chip maximisation only: it cannot represent multi-hand strategic effects (image, trapping across hands), which the superhuman systems likewise ignored and which poker folklore overrates. Within these limits the agent provides the experiment’s cleanest test of a single question: *how far does pure trial and error go, in a game where the environment pushes back?*

8 Experimental Study: Five Paradigms at One Table

The theory of Sections 4–7 makes competing predictions. Rule-based play should be transparently exploitable; equilibrium play should be hard to exploit; learning should adapt to whatever field it faces. This section tests those predictions in silico: the five paradigms share one table in the engine described throughout the paper, and we measure win rates, aggression, and show-down statistics over 100,000 hands of No-Limit Hold’em at blinds of 5/10 with 100-big-blind stacks.

8.1 Setup and methodology

Agents. All five agents run against the same engine API and the same shared abstraction (position class, street, card bucket, betting pattern; four abstract actions). The *rule* agent instantiates Algorithm 1. The *Monte Carlo* agent estimates equity by 300 simulated runouts per decision (the batch budget of Figure 4) and decides by pot odds with safety margins. The *CFR (tabular)* agent plays the pooled average strategy of an external-sampling MCCFR self-play blueprint: 888,361 iterations over 25 minutes on the six-player abstract game, covering 72,741 information-set instances that collapse to 13,873 distinct pooled keys. The *Deep CFR* agent plays the miniature advantage network of Section 6.4 ($14 \rightarrow 32 \rightarrow 4$, reservoir of 60,000 samples, validation correlation 0.44). The *RL* agent plays the tabular Q-function of Section 7 (300,000 episodes of self-play against a mixed pool, 6,585 learned states, $\varepsilon = 0.02$ at play).

Design. The study runs five blocks of 20,000 hands. Each block seats the five paradigms plus a sixth *shadow* seat duplicating one paradigm (block k duplicates paradigm k), so that every paradigm faces every field composition; the shadow’s results are excluded from all statistics, leaving exactly 100,000 counted hands per paradigm. Seat assignment rotates by button-relative distance, so each paradigm occupies every position (button through big blind) uniformly. Stacks are reset to 100 bb before every hand, which makes hands i.i.d. samples: per-paradigm win-rate estimates carry classical standard errors with no wealth-compounding correlation, and block effects can only enter through field composition, which the design varies deliberately. Deals are seeded and reproducible.

Engine correctness and fairness. Three classes of invariant are asserted during play and three were verified offline before the study. In-play: chip conservation was asserted after every hand (total violated assertions over 10^5 hands: 0) and every agent action was validated against the legal-action set before application (illegal actions: 0). Offline, in a prior audit: the dealer was verified card-by-card against an independent replication of the shuffle and deal logic (0

Table 2: Pooled results over 100,000 hands per paradigm (five 20,000-hand blocks, rotating shadow seat; blinds 5/10; 100 bb stacks, reset every hand). Win rate in big blinds per 100 hands; $z = \hat{\mu}/\text{SE}$. VIP: hands voluntarily played; PFR: hands raised pre-flop; aggression: raise frequency over all decisions; SD: showdown frequency; SDW: showdown win rate.

Agent	Win rate			Style fingerprint			Showdown	
	bb/100	SE	z	VPIP %	PFR %	Aggr %	SD %	SDW %
Deep CFR	+144.7	21.0	+6.9	62.0	58.7	56.5	37.2	47.0
Rule-based	+79.3	21.0	+3.8	60.9	3.2	10.8	49.4	44.2
RL (Q-learn)	−26.2	5.2	−5.1	6.3	4.7	8.8	2.5	51.4
Monte Carlo	−51.9	7.1	−7.3	25.2	12.9	15.6	5.0	59.2
CFR (tabular)	−170.0	17.6	−9.7	58.6	46.6	40.6	25.7	47.5

mismatches over 10^5 hands); showdown winners were recomputed by a separate evaluator (0 disagreements); and side-pot awards were re-derived arithmetically (0 violations). Identical agents assigned to different seats showed no seat-dependent win-rate bias beyond multiple-testing noise. We therefore attribute observed differences to the agents.

Statistics. Each paradigm’s per-hand chip result X has mean μ and variance σ^2 estimated from the 10^5 samples; we report $\hat{\mu}$ in big blinds per 100 hands with standard error $\sigma/\sqrt{10^5}$ and 95% confidence intervals. Pairwise differences of independent agents use the sphericity-free two-sample variance; with five agents there are 10 pairwise comparisons, so isolated z -scores near 2 should be read cautiously.

8.2 Results

Table 2 reports the pooled statistics; Figure 6 displays the win rates with confidence intervals, Figure 7 the style fingerprints, and Figure 8 the per-block breakdown.

Three headline facts emerge. First, *every* win rate differs from zero significantly, and the pairwise separations are large: Deep CFR beats the tabular CFR agent by 314.7 bb/100 ($z = 11.5$) and beats the rule agent by 65.4 bb/100 ($z = 2.2$); the rule agent beats the tabular CFR agent by 249.3 bb/100 ($z = 9.1$). Second, the field is extraordinarily showdown-heavy: 64.0% of hands reach showdown (against roughly 30% in tight human six-max play), confirming that these agents neither fold enough nor steal enough. Third, per-hand volatility dwarfs the means—the standard deviation of a single hand’s result ranges from 16 bb (RL) to 67 bb (rule, Deep CFR)—so the 10^5 -hand budget is what makes the ordering statistically decisive, precisely the regime (11) anticipated.

8.3 Style analysis

Figure 7 shows why the win-rate ordering is not a story about aggression alone. The two CFR-family agents occupy nearly the same loose-aggressive coordinates—CFR at (VPIP 58.6%, PFR 46.6%), Deep CFR at (62.0%, 58.7%)—yet their win rates differ by 314.7 bb/100. The rule agent is their photographic negative: almost identical VPIP (60.9%) but near-zero PFR (3.2%), a textbook calling station that nonetheless wins 79.3 bb/100. The RL agent’s learned policy is an extreme nit (VPIP 6.3%), folding everything except monsters—yet its showdown win rate when it does arrive (51.4%) is the field’s best, and its losses are dominated by blind attrition. The Monte Carlo agent plays a tight-preflop, pot-odds-driven style that folds the flop to aggression

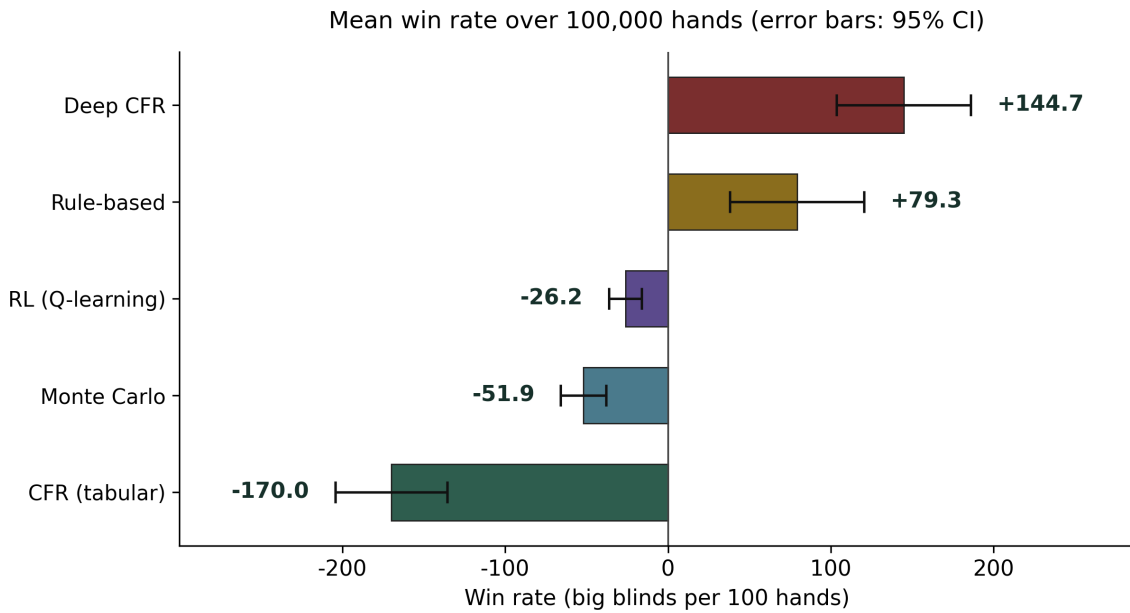


Figure 6: Mean win rate per paradigm over 100,000 hands with 95% confidence intervals. Every agent's win rate differs from zero significantly; the ordering is Deep CFR > rule > RL > Monte Carlo > tabular CFR, with the top and bottom agents separated by 11.5 standard errors.

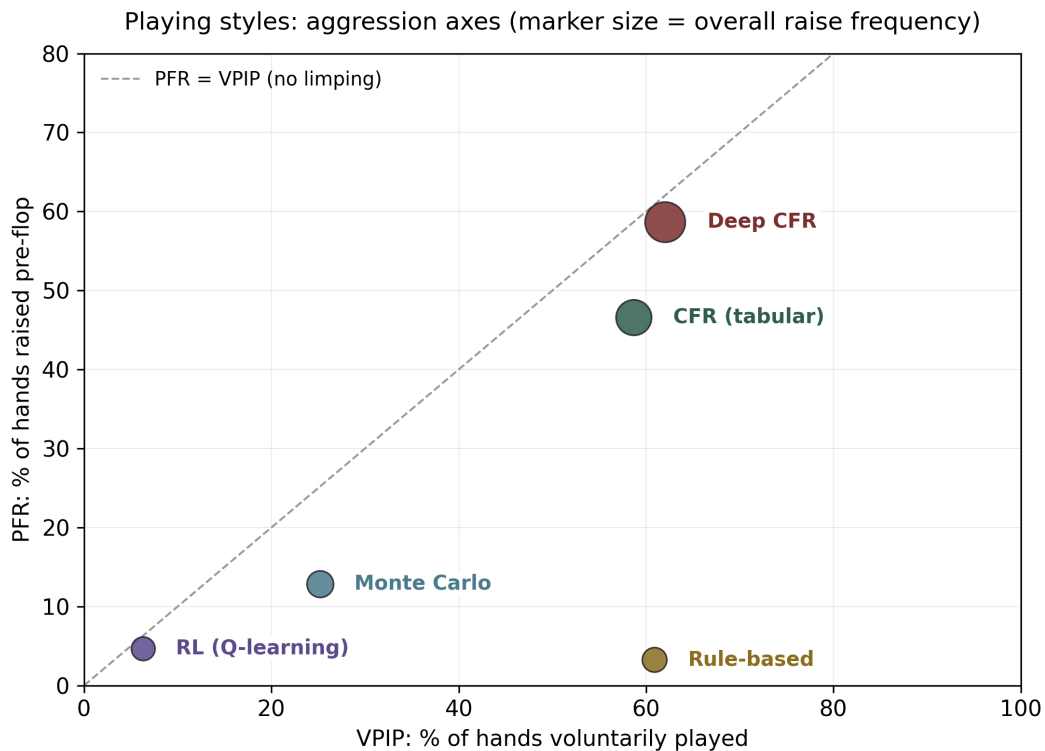


Figure 7: Style fingerprints in the VPIP–PFR plane (marker size proportional to overall raise frequency). The two CFR-family agents sit in the loose-aggressive quadrant with opposite fortunes; the rule agent is a loose-passive calling station; the RL agent learned an extreme nit profile; the Monte Carlo agent is tight and showdown-averse.

(showdown frequency only 5.0%) and collects when it does continue (59.2% at showdown), a profile that is sound against balanced opponents and systematically wrong against a field that bets far too often.

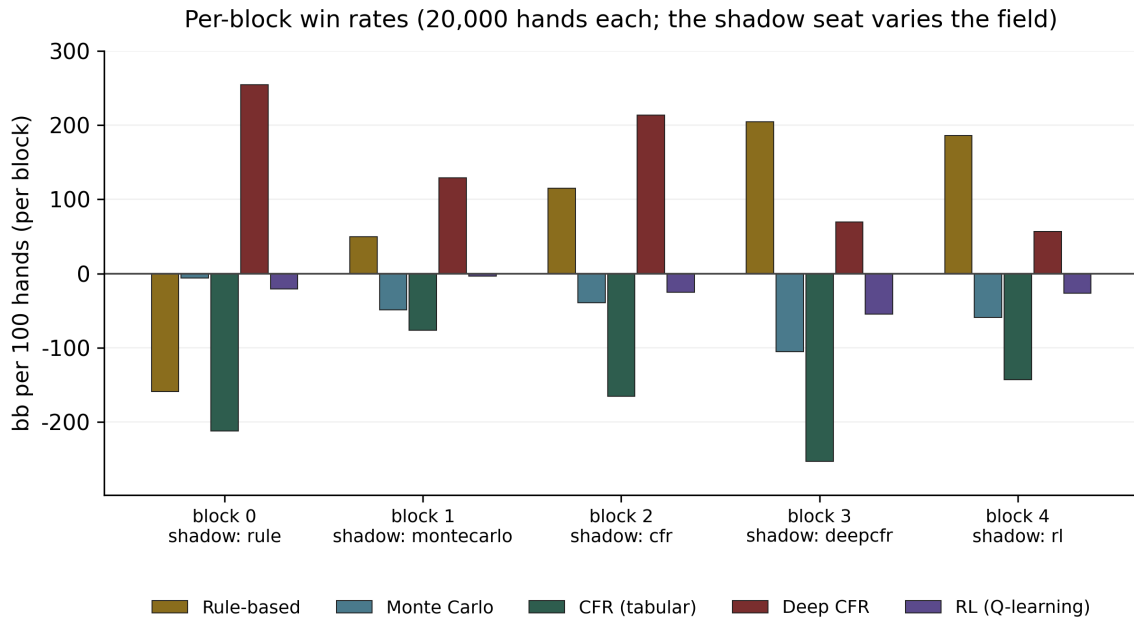


Figure 8: Per-block win rates (20,000 hands per block). The shadow seat’s identity (labelled below each block) changes the field composition; the ordering is stable across all five compositions except for the rule agent, whose results swing with the number of loose-aggressive opponents feeding it calls.

The per-block view (Figure 8) adds a robustness dimension: Deep CFR is the top or near-top earner in every field composition; the tabular CFR agent loses in all five; and the rule agent’s profits concentrate in the blocks whose shadow seats add loose-aggressive opponents to exploit.

8.4 Interpretation: why did the equilibrium agent lose?

The tabular CFR agent’s -170 bb/100 is the study’s most instructive number, because it *looks* like a refutation of Section 6 and is in fact a layered confirmation of the theory’s boundary conditions.

First, the multiplayer void. CFR’s guarantee—average strategy approaches an ϵ -Nash equilibrium whose exploitability bound (16) protects the holder—is a two-player zero-sim statement (Theorem 6.2 in conjunction with Section 5.2). In a six-player game, regret minimisation against one’s own self-play distribution converges toward a coarse correlated equilibrium-like object with *no* safety property: Section 5.2’s warning that multiplayer equilibrium is a robustness heuristic rather than an optimality certificate applies with full force to the blueprint we trained.

Second, the field calls. Equilibrium aggression is priced for a rational responder: in Kuhn poker, the 1:3 bluff-value ratio works because the opponent folds exactly often enough (Proposition 5.1). This field does not fold—it voluntarily plays 60% of hands and reaches showdown 64% of the time—so bluffs at “balanced” frequencies become direct donations, and the blueprint’s PFR of 46.6% feeds pots to opponents who will pay to see rivers. The rule agent’s profit is the mirror image of the same mispricing: a strategy that never bluffs and always calls is maximally

punitive against an over-bluffing field, exactly as the Kuhn arithmetic predicts when the caller’s folding rate deviates from equilibrium.

Third, the table is only partially converged. Diagnostics on the trained blueprint quantify the practical gap between “CFR in principle” and “this CFR run”: 18.7% of the 13,873 pooled strategy rows remain near-uniform (maximum action probability below 0.40), and pre-flop rows—where most chips enter—are the least converged street (mean maximum probability 0.494, against 0.64–0.68 postflop). Whenever play reaches such a row, the agent acts near-uniformly over its legal actions: a sophisticated random player in exactly the spots that this field visits most. Months of compute close this gap in research systems; 25 minutes does not.

Fourth, the sibling controls for the paradigm. The decisive comparison is internal to the CFR family: same abstraction, same engine, same field, same training lineage—yet the tabular agent loses 170 bb/100 while the Deep CFR miniature wins 144.7 bb/100. The difference is not the learning principle but the *representational fallback*: where the table has not been visited, it plays uniformly at random, whereas the advantage network outputs a smooth function of the state features and generalises to unvisited states. In a field this loose, coherence in the long tail of information sets is worth more than sharpness in the well-visited ones. We regard this as the study’s cleanest lesson for practitioners: *in small-budget settings, generalisation across information sets dominates table completeness.*

The RL agent’s result completes the pattern. Its learned policy is a degenerate local optimum—folding all but 6% of hands minimises variance against a training pool that included a random donor, and the terminal-only reward signal offered no gradient away from risk aversion. The policy was near-optimal *against its pool* (training reward +3.65 chips per hand) and loses −26.2 bb/100 against a different field: exploitation learned is field-specific, the guarantee asymmetry of Section 5.2 in miniature, produced by twenty seconds of self-play rather than by design.

8.5 Threats to validity

Four caveats bound the conclusions. (i) The study ranks *implementations with these training budgets*, not paradigms: a months-scale blueprint, or a blueprint evaluated heads-up against a best responder, would occupy a different point in the space, and nothing here contradicts the exploitability guarantees that hold in two-player zero-sum settings. (ii) Field composition is endogenous: win rates are properties of the five-agent ecosystem, and adding one strong tight-aggressive agent would re-price every strategy in the table (the per-block swings of the rule agent make this visible). (iii) The shared abstraction, held fixed for comparability, is coarse (20 preflop and 10 postflop buckets, four bet sizes); all agents inherit its blind spots, and the Deep CFR agent’s advantage is partly that its network interpolates across them. (iv) Statistical resolution, while sufficient for the headline separations (all $|z| \geq 2.2$; the top–bottom gap is 11.5σ), cannot order agents within a few bb/100 of each other. With those boundaries stated, the study’s summary is the one the theory foreshadowed: *safety guarantees are field-independent but payoff-free against weak opponents; exploitation is field-relative and pays exactly as well as the field is exploitable; and representational generalisation decides who survives the long tail of states that training never reached.*

9 Conclusion

This paper traced a single arc: from the arithmetic of cards to agents that play. The probabilistic core (Section 3) is elementary but not optional—pot odds, the $1/\sqrt{n}$ wall of Monte Carlo estimation, and the variance computation behind (11) set the boundaries within which every poker decision system must operate. The game-theoretic core (Sections 5–6) supplies the conceptual shift that separates poker AI from classical game AI: in imperfect-information games,

safety means equilibrium, equilibrium means mixed strategies, and mixed strategies mean accepting negative-EV actions (bluffs, calls that lose on average) as the premium paid for being unreadable. Kuhn poker compresses the entire lesson into twelve information sets, and its exact bluff-value arithmetic—one bluff per three value bets, matching the caller’s pot odds to four decimal places—is the same arithmetic that governs no-limit bet sizing at any depth of stack.

The algorithmic ladder then formalised a trade-off that the experimental study made concrete. Rule-based expert systems (Section 4) are transparent and fast but statically exploitable; Monte Carlo agents sharpen estimation while inheriting static decision thresholds; equilibrium blueprints computed by sampled CFR (Section 6) trade exploitability for unexploitability at the cost of an abstraction’s fidelity; function approximation (Deep CFR) and trial-and-error learning (Q-learning, Section 7) relax the tabular bottleneck in exchange for approximation error and non-stationarity hazards. The 100,000-hand tournament of Section 8 measured where these trade-offs land when five paradigms share one table, under conditions—fixed stacks, rotating positions, seeded deals—designed so that observed differences are attributable to the agents rather than to seat luck, wealth compounding, or measurement artifacts.

Three engineering lessons generalise beyond poker. First, *measurement discipline precedes interpretation*: our own investigation began with an apparent positional bias that dissolved under correct accounting, a reminder that in high-variance domains a plausible statistic unaccompanied by a variance calculation is indistinguishable from noise. Second, *abstraction is not merely compression but a design language*: the choice of buckets, bet sizes, and betting-pattern keys encodes hypotheses about which strategic distinctions matter, and a shared abstraction is what makes controlled comparisons between paradigms meaningful at all. Third, *guarantees and performance are different currencies*: the equilibrium-flavoured agents are certifiably hard to exploit within their abstract game, yet the tournament’s winner is whichever agent’s inductive biases happen to fit the field—a separation that mirrors the exploitation-versus-safety tension the theory predicts, and that future systems reconcile by combining both, as the superhuman programs do with blueprint equilibrium plus targeted real-time search.

The natural extensions of this work follow the field’s trajectory: deeper stacks and finer action abstractions; exploitability measurement of the trained blueprints against best responses computed in the abstract game; end-to-end deep RL with NFSP-style averaging to place the learning paradigm on an equal representational footing with Deep CFR; and opponent modelling layered on top of equilibrium baselines, which is where commercial poker tools now concentrate their effort. The game has been kind to its students: each time the community declared the frontier closed—limit solved, heads-up solved, multiplayer superhuman—the remaining imperfections proved interesting enough to open the next one. Card probabilities, regret, and reinforcement learning will keep finding work at that frontier.

References

- [1] D. Billings, A. Davidson, J. Schaeffer, and D. Szafron. The challenge of poker. *Artificial Intelligence*, 134(1–2):201–250, 2002.
- [2] M. Bowling, N. Burch, M. Johanson, and O. Tammelin. Heads-up limit hold’em poker is solved. *Science*, 347(6218):145–149, 2015.
- [3] N. Brown and T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.
- [4] N. Brown and T. Sandholm. Depth-limited solving for imperfect-information games. In *Advances in Neural Information Processing Systems 31*, pages 7662–7672, 2018.
- [5] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019.

- [6] N. Brown, A. Lerer, S. Gross, and T. Sandholm. Deep counterfactual regret minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 793–802, 2019.
- [7] N. Cesa-Bianchi and G. Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- [8] J. Heinrich and D. Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.
- [9] J. L. Kelly, Jr. A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926, 1956.
- [10] H. W. Kuhn. Extensive games and the problem of information. In *Contributions to the Theory of Games II*, Annals of Mathematics Studies 28, pages 193–216. Princeton University Press, 1953.
- [11] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling. Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems 22*, pages 1078–1086, 2009.
- [12] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017.
- [13] J. F. Nash. Equilibrium points in n -person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49, 1950.
- [14] J. Robinson. An iterative method of solving a game. *Annals of Mathematics*, 54(2):296–301, 1951.
- [15] T. Sandholm. Abstraction for solving imperfect-information games. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence*, pages 4121–4126, 2015.
- [16] D. Schnizlein, M. Bowling, and D. Szafron. Probabilistic card abstraction for equilibrium analysis in poker. In *Proceedings of the 26th AAAI Conference on Artificial Intelligence*, pages 1508–1514, 2012.
- [17] D. Silver, J. Schrittwieser, K. Simonyan, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [18] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.
- [19] O. Tammelin. Solving large imperfect information games using CFR⁺. *arXiv preprint arXiv:1407.5042*, 2014.
- [20] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- [21] K. Waugh, D. Schnizlein, M. Bowling, and D. Szafron. Abstraction pathologies in extensive games. In *Proceedings of the 8th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 781–788, 2009.
- [22] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [23] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20*, pages 1729–1736, 2008.